Version Notice

This documentation is ahead of the last release by 1 commit. You may see documentation for features not yet supported in the latest release v0.0.23 2025-02-07.

Graphs

Don't use a nail gun unless you need a nail gun

If PydanticAI agents are a hammer, and multi-agent workflows are a sledgehammer, then graphs are a nail gun:

- sure, nail guns look cooler than hammers
- but nail guns take a lot more setup than hammers
- and nail guns don't make you a better builder, they make you a builder with a nail gun
- Lastly, (and at the risk of torturing this metaphor), if you're a fan of medieval tools like mallets and untyped Python, you probably won't like nail guns or our approach to graphs. (But then again, if you're not a fan of type hints in Python, you've probably already bounced off PydanticAI to use one of the toy agent frameworks — good luck, and feel free to borrow my sledgehammer when you realize you need it)

In short, graphs are a powerful tool, but they're not the right tool for every job. Please consider other multi-agent approaches before proceeding.

If you're not confident a graph-based approach is a good idea, it might be unnecessary.

Graphs and finite state machines (FSMs) are a powerful abstraction to model, execute, control and visualize complex workflows.

Alongside PydanticAI, we've developed `pydantic-graph` — an async graph and state machine library for Python where nodes and edges are defined using type hints.

While this library is developed as part of PydanticAI; it has no dependency on `pydantic-ai` and can be considered as a pure graph-based state machine library. You may find it useful whether or not you're using PydanticAI or even building with GenAI.

`pydantic-graph` is designed for advanced users and makes heavy use of Python generics and types hints. It is not designed to be as beginner-friendly as PydanticAI.

Very Early beta

Graph support was introduced in v0.0.19 and is in very earlier beta. The API is subject to change. The documentation is incomplete. The implementation is incomplete.

Installation

`pydantic-graph` is a required dependency of `pydantic-ai`, and an optional dependency of `pydantic-ai-slim`, see installation instructions for more information. You can also install it directly:

pip

```
pip install pydantic-graph
```

uv

```
uv add pydantic-graph
```

Graph Types

`pydantic-graph` made up of a few key components:

GraphRunContext

`GraphRunContext` — The context for the graph run, similar to PydanticAI's `RunContext`. This holds the state of the graph and dependencies and is passed to nodes when they're run.

`GraphRunContext` is generic in the state type of the graph it's used in, `StateT`.

End

`End` — return value to indicate the graph run should end.

`End` is generic in the graph return type of the graph it's used in, `RunEndT`.

Nodes

Subclasses of `BaseNode` define nodes for execution in the graph.

Nodes, which are generally `dataclass`es, generally consist of:

- fields containing any parameters required/optional when calling the node
- the business logic to execute the node, in the `run` method
- return annotations of the `run` method, which are read by `pydantic-graph` to determine the outgoing edges of the node

Nodes are generic in:

- **state**, which must have the same type as the state of graphs they're included in, `StateT` has a default of `None`, so if you're not using state you can omit this generic parameter, see stateful graphs for more information
- **deps**, which must have the same type as the deps of the graph they're included in, `DepsT` has a default of `None`, so if you're not using deps you can omit this generic parameter, see dependency injection for more information
- **graph return type** — this only applies if the node returns `End`. `RunEndT` has a default of Never so this generic parameter can be omitted if the node doesn't return `End`, but must be included if it does.

Here's an example of a start or intermediate node in a graph — it can't end the run as it doesn't return `End`:

intermediate_node.py

```python
from dataclasses import dataclass

from pydantic_graph import BaseNode, GraphRunContext


@dataclass
class MyNode(BaseNode[MyState]):  # (1)
    foo: int  # (2)

    async def run(
        self,
        ctx: GraphRunContext[MyState],  # (3)
    ) -> AnotherNode:  # (4)
        ...
        return AnotherNode()
```

1. State in this example is `MyState` (not shown), hence `BaseNode` is parameterized with `MyState`. This node can't end the run, so the `RunEndT` generic parameter is omitted and defaults to `Never`.
2. `MyNode` is a dataclass and has a single field `foo`, an `int`.
3. The `run` method takes a `GraphRunContext` parameter, again parameterized with state `MyState`.
4. The return type of the `run` method is `AnotherNode` (not shown), this is used to determine the outgoing edges of the node.

We could extend `MyNode` to optionally end the run if `foo` is divisible by 5:

intermediate_or_end_node.py

```
from dataclasses import dataclass

from pydantic_graph import BaseNode, End, GraphRunContext


@dataclass
class MyNode(BaseNode[MyState, None, int]):  # (1)
    foo: int

    async def run(
        self,
        ctx: GraphRunContext[MyState],
    ) -> AnotherNode | End[int]:  # (2)
        if self.foo % 5 == 0:
            return End(self.foo)
        else:
            return AnotherNode()
```

1. We parameterize the node with the return type (`int` in this case) as well as state. Because generic parameters are positional-only, we have to include `None` as the second parameter representing deps.
2. The return type of the `run` method is now a union of `AnotherNode` and `End[int]`, this allows the node to end the run if `foo` is divisible by 5.

Graph

`Graph` — this is the execution graph itself, made up of a set of node classes (i.e., `BaseNode` subclasses).

`Graph` is generic in:

- **state** the state type of the graph, `StateT`
- **deps** the deps type of the graph, `DepsT`
- **graph return type** the return type of the graph run, `RunEndT`

Here's an example of a simple graph:

graph_example.py

```python
from __future__ import annotations

from dataclasses import dataclass

from pydantic_graph import BaseNode, End, Graph, GraphRunContext


@dataclass
class DivisibleBy5(BaseNode[None, None, int]):  # (1)
    foo: int

    async def run(
        self,
        ctx: GraphRunContext,
    ) -> Increment | End[int]:
        if self.foo % 5 == 0:
            return End(self.foo)
        else:
            return Increment(self.foo)


@dataclass
class Increment(BaseNode):  # (2)
    foo: int

    async def run(self, ctx: GraphRunContext) -> DivisibleBy5:
        return DivisibleBy5(self.foo + 1)


fives_graph = Graph(nodes=[DivisibleBy5, Increment])  # (3)
result, history = fives_graph.run_sync(DivisibleBy5(4))  # (4)
print(result)
#> 5
# the full history is quite verbose (see below), so we'll just print the summary
print([item.data_snapshot() for item in history])
#> [DivisibleBy5(foo=4), Increment(foo=4), DivisibleBy5(foo=5), End(data=5)]
```

1. The `DivisibleBy5` node is parameterized with `None` for the state param and `None` for the deps param as this graph doesn't use state or deps, and `int` as it can end the run.

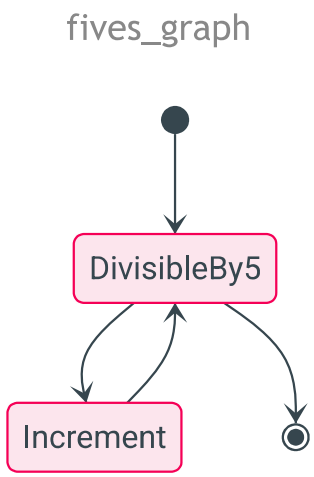
2. The `Increment` node doesn't return `End`, so the `RunEndT` generic parameter is omitted, state can also be omitted as the graph doesn't use state.
3. The graph is created with a sequence of nodes.
4. The graph is run synchronously with `run_sync` the initial state `None` and the start node `DivisibleBy5(4)` are passed as arguments.

(This example is complete, it can be run "as is" with Python 3.10+)

A mermaid diagram for this graph can be generated with the following code:

graph_example_diagram.py

```
from graph_example import DivisibleBy5, fives_graph

fives_graph.mermaid_code(start_node=DivisibleBy5)
```



In order to visualize a graph within a `jupyter-notebook`, `IPython.display` needs to be used:

jupyter_display_mermaid.py

```
from graph_example import DivisibleBy5, fives_graph
from IPython.display import Image, display

display(Image(fives_graph.mermaid_image(start_node=DivisibleBy5)))
```

Stateful Graphs

The "state" concept in `pydantic-graph` provides an optional way to access and mutate an object (often a `dataclass` or Pydantic model) as nodes run in a graph. If you think of Graphs as a

production line, then your state is the engine being passed along the line and built up by each node as the graph is run.

In the future, we intend to extend `pydantic-graph` to provide state persistence with the state recorded after each node is run, see #695.

Here's an example of a graph which represents a vending machine where the user may insert coins and select a product to purchase.

vending_machine.py

```
from __future__ import annotations

from dataclasses import dataclass

from rich.prompt import Prompt

from pydantic_graph import BaseNode, End, Graph, GraphRunContext


@dataclass
class MachineState:  1
    user_balance: float = 0.0
    product: str | None = None


@dataclass
class InsertCoin(BaseNode[MachineState]):  3
    async def run(self, ctx: GraphRunContext[MachineState]) -> CoinsInserted:
16
        return CoinsInserted(float(Prompt.ask('Insert coins')))  4


@dataclass
class CoinsInserted(BaseNode[MachineState]):
    amount: float  5

    async def run(
        self, ctx: GraphRunContext[MachineState]
    ) -> SelectProduct | Purchase:  17
        ctx.state.user_balance += self.amount  6
        if ctx.state.product is not None:  7
            return Purchase(ctx.state.product)
        else:
            return SelectProduct()


@dataclass
class SelectProduct(BaseNode[MachineState]):
    async def run(self, ctx: GraphRunContext[MachineState]) -> Purchase:
        return Purchase(Prompt.ask('Select product'))
```

```
PRODUCT_PRICES = {  2
    'water': 1.25,
    'soda': 1.50,
    'crisps': 1.75,
    'chocolate': 2.00,
}


@dataclass
class Purchase(BaseNode[MachineState, None, None]):  18
    product: str

    async def run(
        self, ctx: GraphRunContext[MachineState]
    ) -> End | InsertCoin | SelectProduct:
        if price := PRODUCT_PRICES.get(self.product):  8
            ctx.state.product = self.product  9
            if ctx.state.user_balance >= price:  10
                ctx.state.user_balance -= price
                return End(None)
            else:
                diff = price - ctx.state.user_balance
                print(f'Not enough money for {self.product}, need {diff:0.2f}
more')
                #> Not enough money for crisps, need 0.75 more
                return InsertCoin()  11
        else:
            print(f'No such product: {self.product}, try again')
            return SelectProduct()  12


vending_machine_graph = Graph(  13
    nodes=[InsertCoin, CoinsInserted, SelectProduct, Purchase]
)


async def main():
    state = MachineState()  14
    await vending_machine_graph.run(InsertCoin(), state=state)  15
    print(f'purchase successful item={state.product} change=
{state.user_balance:0.2f}')
    #> purchase successful item=crisps change=0.25
```

1. The state of the vending machine is defined as a dataclass with the user's balance and the product they've selected, if any.
2. A dictionary of products mapped to prices.
3. The `InsertCoin` node, `BaseNode` is parameterized with `MachineState` as that's the state used in this graph.

4. The `InsertCoin` node prompts the user to insert coins. We keep things simple by just entering a monetary amount as a float. Before you start thinking this is a toy too since it's using rich's `Prompt.ask` within nodes, see below for how control flow can be managed when nodes require external input.
5. The `CoinsInserted` node; again this is a `dataclass`, in this case with one field `amount`, thus nodes calling `CoinsInserted` must provide an amount.
6. Update the user's balance with the amount inserted.
7. If the user has already selected a product, go to `Purchase`, otherwise go to `SelectProduct`.
8. In the `Purchase` node, look up the price of the product if the user entered a valid product.
9. If the user did enter a valid product, set the product in the state so we don't revisit `SelectProduct`.
10. If the balance is enough to purchase the product, adjust the balance to reflect the purchase and return `End` to end the graph. We're not using the run return type, so we call `End` with `None`.
11. If the balance is insufficient, to go `InsertCoin` to prompt the user to insert more coins.
12. If the product is invalid, go to `SelectProduct` to prompt the user to select a product again.
13. The graph is created by passing a list of nodes to `Graph`. Order of nodes is not important, but will alter how diagrams are displayed.
14. Initialize the state. This will be passed to the graph run and mutated as the graph runs.
15. Run the graph with the initial state. Since the graph can be run from any node, we must pass the start node — in this case, `InsertCoin`. `Graph.run` returns a tuple of the return value (`None`) in this case, and the history of the graph run.
16. The return type of the node's `run` method is important as it is used to determine the outgoing edges of the node. This information in turn is used to render mermaid diagrams and is enforced at runtime to detect misbehavior as soon as possible.
17. The return type of `CoinsInserted`'s `run` method is a union, meaning multiple outgoing edges are possible.
18. Unlike other nodes, `Purchase` can end the run, so the `RunEndT` generic parameter must be set. In this case it's `None` since the graph run return type is `None`.

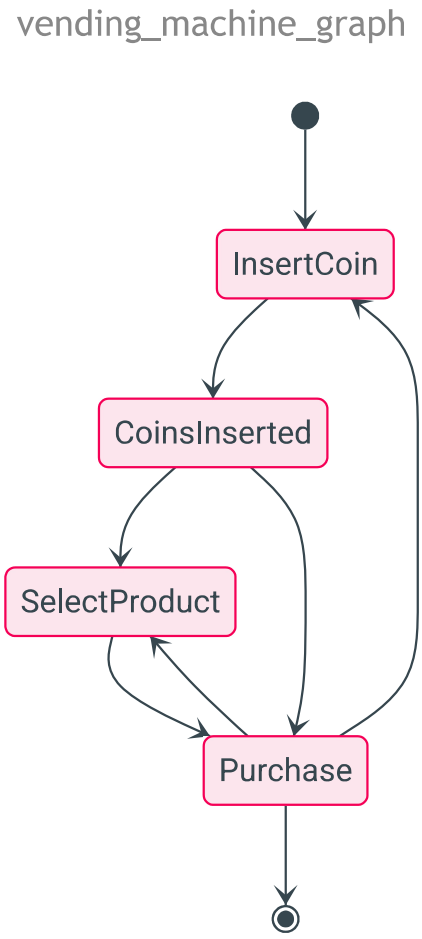
(This example is complete, it can be run "as is" with Python 3.10+ — you'll need to add `asyncio.run(main())` to run `main`)

A mermaid diagram for this graph can be generated with the following code:

vending_machine_diagram.py

```
from vending_machine import InsertCoin, vending_machine_graph

vending_machine_graph.mermaid_code(start_node=InsertCoin)
```

The diagram generated by the above code is:



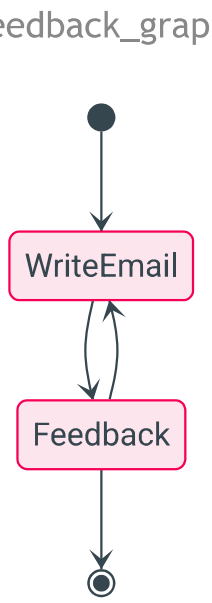
See below for more information on generating diagrams.

GenAI Example

So far we haven't shown an example of a Graph that actually uses PydanticAI or GenAI at all.

In this example, one agent generates a welcome email to a user and the other agent provides feedback on the email.

This graph has a very simple structure:



genai_email_feedback.py

```python
from __future__ import annotations as _annotations

from dataclasses import dataclass, field

from pydantic import BaseModel, EmailStr

from pydantic_ai import Agent
from pydantic_ai.format_as_xml import format_as_xml
from pydantic_ai.messages import ModelMessage
from pydantic_graph import BaseNode, End, Graph, GraphRunContext


@dataclass
class User:
    name: str
    email: EmailStr
    interests: list[str]


@dataclass
class Email:
    subject: str
    body: str


@dataclass
class State:
    user: User
    write_agent_messages: list[ModelMessage] = field(default_factory=list)
```

```python
email_writer_agent = Agent(
    'google-vertex:gemini-1.5-pro',
    result_type=Email,
    system_prompt='Write a welcome email to our tech blog.',
)


@dataclass
class WriteEmail(BaseNode[State]):
    email_feedback: str | None = None

    async def run(self, ctx: GraphRunContext[State]) -> Feedback:
        if self.email_feedback:
            prompt = (
                f'Rewrite the email for the user:\n'
                f'{format_as_xml(ctx.state.user)}\n'
                f'Feedback: {self.email_feedback}'
            )
        else:
            prompt = (
                f'Write a welcome email for the user:\n'
                f'{format_as_xml(ctx.state.user)}'
            )

        result = await email_writer_agent.run(
            prompt,
            message_history=ctx.state.write_agent_messages,
        )
        ctx.state.write_agent_messages += result.all_messages()
        return Feedback(result.data)


class EmailRequiresWrite(BaseModel):
    feedback: str


class EmailOk(BaseModel):
    pass


feedback_agent = Agent[None, EmailRequiresWrite | EmailOk](
    'openai:gpt-4o',
    result_type=EmailRequiresWrite | EmailOk,  # type: ignore
    system_prompt=(
        'Review the email and provide feedback, email must reference the users
specific interests.'
    ),
)


@dataclass
class Feedback(BaseNode[State, None, Email]):
```

```
    email: Email

    async def run(
        self,
        ctx: GraphRunContext[State],
    ) -> WriteEmail | End[Email]:
        prompt = format_as_xml({'user': ctx.state.user, 'email': self.email})
        result = await feedback_agent.run(prompt)
        if isinstance(result.data, EmailRequiresWrite):
            return WriteEmail(email_feedback=result.data.feedback)
        else:
            return End(self.email)


async def main():
    user = User(
        name='John Doe',
        email='john.joe@example.com',
        interests=['Haskel', 'Lisp', 'Fortran'],
    )
    state = State(user)
    feedback_graph = Graph(nodes=(WriteEmail, Feedback))
    email, _ = await feedback_graph.run(WriteEmail(), state=state)
    print(email)
    """
    Email(
        subject='Welcome to our tech blog!',
        body='Hello John, Welcome to our tech blog! ...',
    )
    """
```

(This example is complete, it can be run "as is" with Python 3.10+ — you'll need to add `asyncio.run(main())` to run `main`)

Custom Control Flow

In many real-world applications, Graphs cannot run uninterrupted from start to finish — they might require external input, or run over an extended period of time such that a single process cannot execute the entire graph run from start to finish without interruption.

In these scenarios the `next` method can be used to run the graph one node at a time.

In this example, an AI asks the user a question, the user provides an answer, the AI evaluates the answer and ends if the user got it right or asks another question if they got it wrong.

`ai_q_and_a_graph.py` — `question_graph` definition

ai_q_and_a_graph.py

```python
from __future__ import annotations as _annotations

from dataclasses import dataclass, field

from pydantic_graph import BaseNode, End, Graph, GraphRunContext

from pydantic_ai import Agent
from pydantic_ai.format_as_xml import format_as_xml
from pydantic_ai.messages import ModelMessage

ask_agent = Agent('openai:gpt-4o', result_type=str)


@dataclass
class QuestionState:
    question: str | None = None
    ask_agent_messages: list[ModelMessage] = field(default_factory=list)
    evaluate_agent_messages: list[ModelMessage] = field(default_factory=list)


@dataclass
class Ask(BaseNode[QuestionState]):
    async def run(self, ctx: GraphRunContext[QuestionState]) -> Answer:
        result = await ask_agent.run(
            'Ask a simple question with a single correct answer.',
            message_history=ctx.state.ask_agent_messages,
        )
        ctx.state.ask_agent_messages += result.all_messages()
        ctx.state.question = result.data
        return Answer(result.data)


@dataclass
class Answer(BaseNode[QuestionState]):
    question: str
    answer: str | None = None

    async def run(self, ctx: GraphRunContext[QuestionState]) -> Evaluate:
        assert self.answer is not None
        return Evaluate(self.answer)


@dataclass
class EvaluationResult:
    correct: bool
    comment: str


evaluate_agent = Agent(
    'openai:gpt-4o',
    result_type=EvaluationResult,
```

```
    system_prompt='Given a question and answer, evaluate if the answer is
correct.',
)


@dataclass
class Evaluate(BaseNode[QuestionState]):
    answer: str

    async def run(
        self,
        ctx: GraphRunContext[QuestionState],
    ) -> End[str] | Reprimand:
        assert ctx.state.question is not None
        result = await evaluate_agent.run(
            format_as_xml({'question': ctx.state.question, 'answer': self.answer}),
            message_history=ctx.state.evaluate_agent_messages,
        )
        ctx.state.evaluate_agent_messages += result.all_messages()
        if result.data.correct:
            return End(result.data.comment)
        else:
            return Reprimand(result.data.comment)


@dataclass
class Reprimand(BaseNode[QuestionState]):
    comment: str

    async def run(self, ctx: GraphRunContext[QuestionState]) -> Ask:
        print(f'Comment: {self.comment}')
        ctx.state.question = None
        return Ask()


question_graph = Graph(nodes=(Ask, Answer, Evaluate, Reprimand))
```

(This example is complete, it can be run "as is" with Python 3.10+)

ai_q_and_a_run.py

```
from rich.prompt import Prompt

from pydantic_graph import End, HistoryStep

from ai_q_and_a_graph import Ask, question_graph, QuestionState, Answer


async def main():
    state = QuestionState()  # (1)
    node = Ask()  # (2)
    history: list[HistoryStep[QuestionState]] = []  # (3)
    while True:
        node = await question_graph.next(node, history, state=state)  # (4)
```

```
        if isinstance(node, Answer):
            node.answer = Prompt.ask(node.question)  # 5
        elif isinstance(node, End):  # 6
            print(f'Correct answer! {node.data}')
            #> Correct answer! Well done, 1 + 1 = 2
            print([e.data_snapshot() for e in history])
            """
            [
                Ask(),
                Answer(question='What is the capital of France?',
answer='Vichy'),
                Evaluate(answer='Vichy'),
                Reprimand(comment='Vichy is no longer the capital of France.'),
                Ask(),
                Answer(question='what is 1 + 1?', answer='2'),
                Evaluate(answer='2'),
            ]
            """
            return
        # otherwise just continue
```

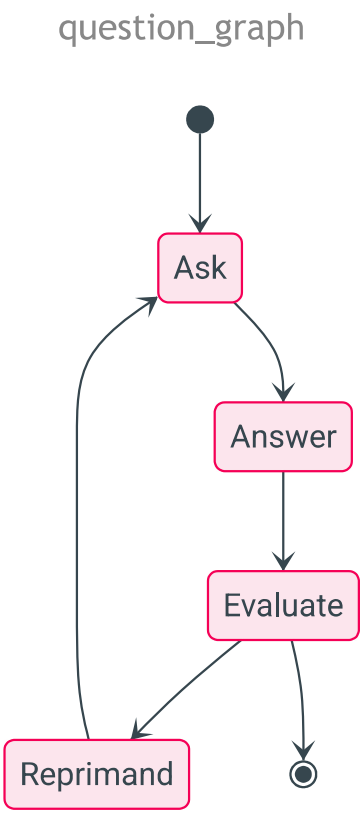
1. Create the state object which will be mutated by `next`.
2. The start node is `Ask` but will be updated by `next` as the graph runs.
3. The history of the graph run is stored in a list of `HistoryStep` objects. Again `next` will update this list in place.
4. Run the graph one node at a time, updating the state, current node and history as the graph runs.
5. If the current node is an `Answer` node, prompt the user for an answer.
6. Since we're using `next` we have to manually check for an `End` and exit the loop if we get one.

(This example is complete, it can be run "as is" with Python 3.10+ — you'll need to add `asyncio.run(main())` to run `main`)

A mermaid diagram for this graph can be generated with the following code:

ai_q_and_a_diagram.py

```
from ai_q_and_a_graph import Ask, question_graph

question_graph.mermaid_code(start_node=Ask)
```



You maybe have noticed that although this examples transfers control flow out of the graph run, we're still using rich's `Prompt.ask` to get user input, with the process hanging while we wait for the user to enter a response. For an example of genuine out-of-process control flow, see the question graph example.

Dependency Injection

As with PydanticAI, `pydantic-graph` supports dependency injection via a generic parameter on `Graph` and `BaseNode`, and the `GraphRunContext.deps` fields.

As an example of dependency injection, let's modify the `DivisibleBy5` example above to use a `ProcessPoolExecutor` to run the compute load in a separate process (this is a contrived example, `ProcessPoolExecutor` wouldn't actually improve performance in this example):

deps_example.py

```
from __future__ import annotations

import asyncio
from concurrent.futures import ProcessPoolExecutor
```

```
from dataclasses import dataclass

from pydantic_graph import BaseNode, End, Graph, GraphRunContext


@dataclass
class GraphDeps:
    executor: ProcessPoolExecutor


@dataclass
class DivisibleBy5(BaseNode[None, None, int]):
    foo: int

    async def run(
        self,
        ctx: GraphRunContext,
    ) -> Increment | End[int]:
        if self.foo % 5 == 0:
            return End(self.foo)
        else:
            return Increment(self.foo)


@dataclass
class Increment(BaseNode):
    foo: int

    async def run(self, ctx: GraphRunContext) -> DivisibleBy5:
        loop = asyncio.get_running_loop()
        compute_result = await loop.run_in_executor(
            ctx.deps.executor,
            self.compute,
        )
        return DivisibleBy5(compute_result)

    def compute(self) -> int:
        return self.foo + 1


fives_graph = Graph(nodes=[DivisibleBy5, Increment])


async def main():
    with ProcessPoolExecutor() as executor:
        deps = GraphDeps(executor)
        result, history = await fives_graph.run(DivisibleBy5(3), deps=deps)
    print(result)
    #> 5
    # the full history is quite verbose (see below), so we'll just print the
summary
    print([item.data_snapshot() for item in history])
    """
```

```
    [
        DivisibleBy5(foo=3),
        Increment(foo=3),
        DivisibleBy5(foo=4),
        Increment(foo=4),
        DivisibleBy5(foo=5),
        End(data=5),
    ]
    """
```

(This example is complete, it can be run "as is" with Python 3.10+ — you'll need to add `asyncio.run(main())` to run `main`)

Mermaid Diagrams

Pydantic Graph can generate mermaid `stateDiagram-v2` diagrams for graphs, as shown above.

These diagrams can be generated with:

- `Graph.mermaid_code` to generate the mermaid code for a graph
- `Graph.mermaid_image` to generate an image of the graph using mermaid.ink
- `Graph.mermaid_save` to generate an image of the graph using mermaid.ink and save it to a file

Beyond the diagrams shown above, you can also customize mermaid diagrams with the following options:

- `Edge` allows you to apply a label to an edge
- `BaseNode.docstring_notes` and `BaseNode.get_note` allows you to add notes to nodes
- The `highlighted_nodes` parameter allows you to highlight specific node(s) in the diagram

Putting that together, we can edit the last `ai_q_and_a_graph.py` example to:

- add labels to some edges
- add a note to the `Ask` node
- highlight the `Answer` node
- save the diagram as a `PNG` image to file

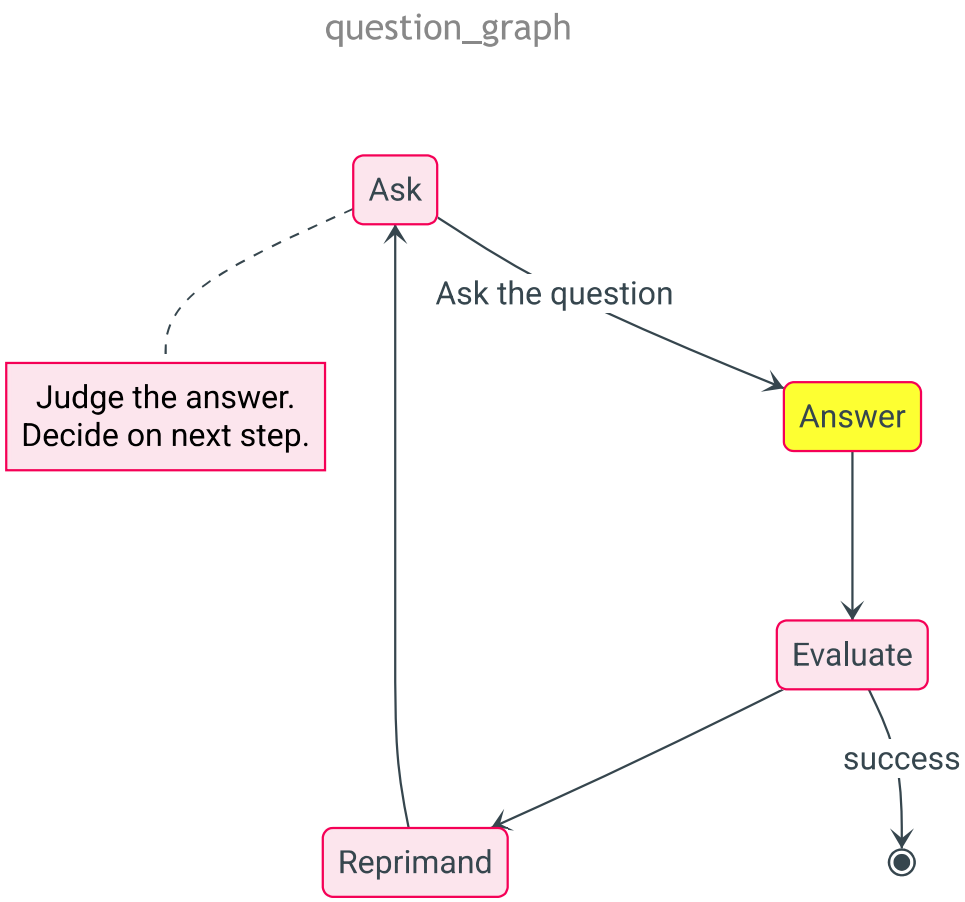
ai_q_and_a_graph_extra.py

```
...
from typing import Annotated

from pydantic_graph import BaseNode, End, Graph, GraphRunContext, Edge
```

```python
...

@dataclass
class Ask(BaseNode[QuestionState]):
    """Generate question using GPT-4o."""
    docstring_notes = True
    async def run(
        self, ctx: GraphRunContext[QuestionState]
    ) -> Annotated[Answer, Edge(label='Ask the question')]:
        ...

...

@dataclass
class Evaluate(BaseNode[QuestionState]):
    answer: str

    async def run(
            self,
            ctx: GraphRunContext[QuestionState],
    ) -> Annotated[End[str], Edge(label='success')] | Reprimand:
        ...

...

question_graph.mermaid_save('image.png', highlighted_nodes=[Answer])
```

(This example is not complete and cannot be run directly)

Would generate and image that looks like this:



Setting Direction of the State Diagram

You can specify the direction of the state diagram using one of the following values:

- `'TB'`: Top to bottom, the diagram flows vertically from top to bottom.
- `'LR'`: Left to right, the diagram flows horizontally from left to right.
- `'RL'`: Right to left, the diagram flows horizontally from right to left.
- `'BT'`: Bottom to top, the diagram flows vertically from bottom to top.

Here is an example of how to do this using 'Left to Right' (LR) instead of the default 'Top to Bottom' (TB)

vending_machine_diagram.py

```
from vending_machine import InsertCoin, vending_machine_graph

vending_machine_graph.mermaid_code(start_node=InsertCoin, direction='LR')
```

vending_machine_graph